

Release Runbook — helix_proxy

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Scope:**

Cutting a project-prefixed release tag (helix_proxy-<version>) on the main repository and every owned submodule that changed, published to all configured remotes via BOTH the GitHub CLI (gh) and the GitLab CLI (glab). **Authority:** Helix Constitution §11.4.151 (project-prefixed release tags/versions), §11.4.113 (absolute no-force-push; fast-forward-only), §2.1 (multi-upstream push), §11.4.40 (full-suite retest before tag), §11.4.126 (release-scope terminal condition), §11.4.44 (revision header), §11.4.65 (universal Markdown export).

This runbook is a read-only planning artefact. It documents the procedure; executing any state-changing step (tag/push/release-create) is a separate, operator-authorised action.

Audit results 2026-07-01

All commands below were run read-only against /run/media/milosvasic/DATA4TB/Projects/helix_proxy on 2026-07-01. No state-changing command was executed. No authentication attempt was made.

1. Remotes — git remote -v

```
github  git@github.com:HelixDevelopment/HelixProxy.git (fetch)
github  git@github.com:HelixDevelopment/HelixProxy.git (push)
origin  git@github.com:HelixDevelopment/HelixProxy.git (fetch)
origin  git@github.com:HelixDevelopment/HelixProxy.git (push)
upstream git@github.com:HelixDevelopment/HelixProxy.git (fetch)
upstream git@github.com:HelixDevelopment/HelixProxy.git (push)
```

- **NO-HTTPS hard rule:** PASS — all three remotes use git@github.com: SSH URLs. No HTTPS remote present.
- **Observation (not a pass/fail, flagged for operator):** all three remote names (github, origin, upstream) are **aliases pointing at the SAME GitHub URL** git@github.com:HelixDevelopment/HelixProxy.git. There is **no GitLab remote configured** for this repository. Under §2.1 (multi-upstream push is the norm) a true multi-mirror setup

would add a distinct GitLab remote. The current configuration pushes to one physical GitHub repo under three names. See OPERATOR-BLOCKED item **OB-1**.

Verdict: READY (SSH-only) — with an OPERATOR note that a GitLab mirror remote is not yet configured.

2. Existing release tags — `git tag -l 'helix_proxy-*`

```
helix_proxy-0.1.0-dev-0.0.1
```

Full tag list (`git tag -l`): `helix_proxy-0.1.0-dev-0.0.1`, `v1.1.0`, `v1.2.0` (3 tags total). Tag detail:

```
helix_proxy-0.1.0-dev-0.0.1 | 2026-07-01 09:26:23 +0300 | tag  
(annotated)
```

- **§11.4.151 naming:** PASS — the most recent release tag `helix_proxy-0.1.0-dev-0.0.1` carries the resolved `helix_proxy-` prefix and is an annotated tag.
- The legacy `v1.1.0` / `v1.2.0` tags predate the prefixed scheme (mirrored in `CHANGELOG.md` as `[1.1.0]` / `[1.2.0]`); they are not release surfaces going forward.
- **Next tag** per monotonic increment: `helix_proxy-0.1.0-dev-0.0.2` (matches the requested target).

Verdict: READY.

3. GitHub CLI — `gh auth status / gh repo view`

```
github.com
```

- ✓ Logged in to github.com account `milos85vasic` (keyring)
- Active account: `true`
- Git operations protocol: `ssh`
- Token scopes: `'admin:public_key', 'gist', 'read:org', 'repo'`

```
repo: {"nameWithOwner": "HelixDevelopment/HelixProxy",  
      "url": "https://github.com/HelixDevelopment/HelixProxy"}
```

- Authenticated as `milos85vasic`, SSH git protocol, repo scope present (sufficient for `gh release create`).
- Resolved repo: `HelixDevelopment/HelixProxy`.

Verdict: READY.

4. GitLab CLI — `glab auth status / glab --version`

```
glab 1.91.0 ()
```

```
gitlab.com
```

- ✓ Logged in to gitlab.com as `milos85vasic` (`GITLAB_TOKEN`)
- ✓ Git operations for gitlab.com configured to use `ssh` protocol.

✓ API calls for gitlab.com are made over https protocol.
✓ REST API Endpoint: https://gitlab.com/api/v4/
! Token is from environment variable GITLAB_TOKEN (takes precedence over config/keyring).

- glab is present (v1.91.0) and **authenticated** to gitlab.com as milos85vasic via the GITLAB_TOKEN environment variable, git protocol SSH.
- **Caveat (honest, per §11.4.6):** the CLI is authed, but **the GitLab project that this repo would publish to is UNKNOWN** — there is no GitLab remote in git remote -v and no .gitlab/project mapping was found in the audit. glab release create needs a --repo <group>/<project> target (or a configured GitLab remote). The token being sourced from GITLAB_TOKEN (env) rather than keyring means it depends on the operator's shell/wrapper environment being present at release time. See OPERATOR-BLOCKED **OB-1** / **OB-2**.

Verdict: PARTIALLY READY — CLI authed; **target GitLab project + persistent auth source are OPERATOR-BLOCKED/UNKNOWN**.

5. Release-prefix resolution (§11.4.151)

.env : DOES NOT EXIST
.env.example : exists, but contains NO
HELIX_RELEASE_PREFIX
project root dir basename : helix_proxy

- Resolution order per §11.4.151: (1) HELIX_RELEASE_PREFIX from .env — **unset (no .env)**; (2) fallback = lowercased snake_case of the project root directory name = **helix_proxy**.
- **Resolved prefix: helix_proxy**. Consistent with the existing helix_proxy-0.1.0-dev-0.0.1 tag.

Verdict: READY (prefix deterministically resolvable with zero operator input). Optional hardening: declare HELIX_RELEASE_PREFIX=helix_proxy in a gitignored .env and document it in the tracked .env.example (§11.4.77) so the prefix is authoritative rather than dir-name-derived.

6. Changelog location + format

CHANGELOG.md : EXISTS at repo root (Keep a Changelog format)
docs/changelogs/ : DOES NOT EXIST
docs/releases/ : created by this audit (holds this runbook)

submodules/*/CHANGELOG.{md,html,pdf} : each owned submodule ships its own

- The canonical changelog is the **root CHANGELOG.md**, "Keep a Changelog" style with **##** [<version>] - <YYYY-MM-DD> sections and **###** Added / **###** Fixed / **###** Changed subsections, plus an **##** [Unreleased] block at the top. Latest documented entries: [1.2.0] - 2026-04-23, [1.1.0] - 2026-04-18, [1.0.0] - 2024-03-26.
- **Gap:** CHANGELOG.md has **no entry for 0.1.0-dev-0.0.1** nor for the upcoming 0.1.0-dev-0.0.2; the newest section is [1.2.0]. The changelog must gain a section for the new release before/at tagging (see procedure step 5). Note the versioning discontinuity between the legacy 1.x semver line and the current 0.1.0-dev-0.0.x prefixed line — reconcile per operator intent (**OB-3**).

Verdict: READY (format known) — with the required action to add the new release section (and, if desired, HTML/PDF exports per §11.4.65).

7. Branch / HEAD / remote-tip alignment

```
current branch : feature/vpn-aware-dynamic-routing
HEAD           : 2812f48c4a0dd31d523b29056702a494b43af48d
working tree   : clean (git status --porcelain -> 0 entries)
```

```
Local + remote refs (all identical @ 2812f48):
refs/heads/main                2812f48
refs/heads/feature/vpn-aware-dynamic-routing 2812f48
refs/remotes/github/main       2812f48
refs/remotes/origin/main       2812f48
refs/remotes/upstream/main     2812f48
github/feature/... origin/feature/... upstream/feature/...
2812f48
```

- **Everything is currently in sync at one commit (2812f48):** local main, local feature branch, and all three remotes' main and feature tips are the same object. Working tree clean.
- The checkout is on the **feature branch** feature/vpn-aware-dynamic-routing, not main. Per §11.4.126 the release-scope tag is cut on main after the feature work lands there. Right now main == feature == remotes, so a fast-forward of main to the feature tip is trivial (or a no-op if already there). Once new work advances the feature branch, land it on main (§11.4.113 FF-only merge) **before** tagging.

Verdict: READY — tree clean, all refs aligned; tag on main.

Consolidated readiness verdict

Prerequisite	Verdict	Note
SSH-only remotes	READY	3 SSH aliases → one GitHub repo
Prefix (§11.4.151)	READY	resolves to helix_proxy (dir fallback)
Existing tag naming	READY	helix_proxy-0.1.0-dev-0.0.1 compliant
GitHub CLI (gh)	READY	authed, repo scope, HelixDevelopment/HelixProxy
GitLab CLI (glab)	PARTIALLY READY	authed; target project UNKNOWN (OB-1/OB-2)
Multi-upstream (§2.1)	OPERATOR-BLOCKED	no GitLab mirror remote configured (OB-1)
Changelog	READY (format)	must add new-release section (OB-3 discontinuity)
Branch/HEAD alignment	READY	clean tree, all refs @ 2812f48; tag on main

Overall: NOT YET FULLY READY to cut helix_proxy-0.1.0-dev-0.0.2 across BOTH GitHub and GitLab. GitHub side is fully READY. The **GitLab side is OPERATOR-BLOCKED** on a known target project + a GitLab mirror remote (OB-1, OB-2). The GitHub-only release can proceed once the §11.4.40 full-retest gate and the changelog entry are complete.

Release procedure

Target next tag: **helix_proxy-0.1.0-dev-0.0.2** (prefix helix_proxy resolved per §11.4.151, monotonic increment from -0.0.1).

Replace <VERSION> with 0.1.0-dev-0.0.2 and <TAG> with helix_proxy-0.1.0-dev-0.0.2 throughout. Do NOT run any of the state-changing steps until the operator authorises the release AND the §11.4.40 gate is GREEN.

Step 0 — Fetch-before-anything (§11.4.37 / §11.4.71 / §11.4.113 step 1)

```
git -C <repo> fetch --all --prune --tags
git -C <repo> submodule foreach --recursive 'git fetch --all --prune --tags --quiet'
```

Confirm HEAD. @{u} is empty for main on every remote; integrate any divergence FF-only before proceeding. Remote state is unknowable without this fetch (§11.4.6).

Step 1 — Land feature work on main (§11.4.126 / §11.4.113)

```
git -C <repo> switch main
git -C <repo> merge --ff-only feature/vpn-aware-dynamic-routing
```

FF-only merge only (§11.4.113: no force, no rebase of a shared branch, no history rewrite). If the merge is not fast-forwardable, use the merge-onto-latest-main integration path of §11.4.113 (fetch → base on latest main → merge → resolve → commit), never a force.

Step 2 — §11.4.40 FULL-SUITE RETEST GATE (mandatory, BEFORE tagging)

Do NOT create the tag until a COMPLETE retest on a clean baseline is GREEN. Per §11.4.40 the complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-target validation cycle, (4) meta-test mutation sweep, (5) Challenge bank full sweep (via submodules/challenges + HelixQA submodules/helix_qa), (6) Issues/Fixed state audit, (7) CONTINUATION sync check. Spot-check retests of only batch-touched tests are FORBIDDEN. Every PASS carries captured evidence (§11.4 / §11.4.69). A red gate blocks the tag.

Step 3 — Update the changelog (§11.4.65)

Add a new section to root CHANGELOG.md, mirroring the Keep-a-Changelog format already in use:

```
## [0.1.0-dev-0.0.2] - <YYYY-MM-DD>
```

```
### Added
```

```
- <user-visible additions>
```

```
### Fixed
```

```
- <bug fixes with root cause>
```

```
### Changed
```

```
- <behavioural / config changes>
```

Move relevant items out of ## [Unreleased]. Refresh HTML/PDF siblings if the project's export tooling is wired (§11.4.65) — the existing CHANGELOG.md has no exports yet at repo root; owned submodules do ship .html/.pdf.

Step 4 — Determine changed owned submodules (§11.4.151 cascade scope)

Owned submodules (SAME-prefix tag applies to each that changed in this release):

Path	Upstream	Org (owned)
constitution	git@github.com:HelixDevelopment/ HelixConstitution.git	HelixDevelopment
submodules/ docs_chain	git@github.com:vasic-digital/ docs_chain.git	vasic-digital
submodules/ challenges	git@github.com:vasic-digital/ Challenges.git	vasic-digital
submodules/ helix_qa	git@github.com:HelixDevelopment/ HelixQA.git	HelixDevelopment
submodules/ containers	git@github.com:vasic-digital/ containers.git	vasic-digital

For each owned submodule, check whether its pointer advanced in this release window (`git -C <repo> submodule status`; compare against the pointer at `helix_proxy-0.1.0-dev-0.0.1`). **Only submodules that changed** get the release tag; untouched submodules stay pinned. See **OB-4** — these submodules currently carry their own `helixcode-v1.1.0-*` describe line, which is a DIFFERENT prefix than `helix_proxy`; whether this project stamps them with a `helix_proxy-<VERSION>` tag or relies on their own release line is an operator decision, not a guess.

Step 5 — Create the prefixed annotated tag on main (§11.4.151)

Main repo (and each changed owned submodule, run from its worktree):

```
# main repo, on main @ the release commit
git -C <repo> tag -a helix_proxy-0.1.0-dev-0.0.2 \
    -m "helix_proxy 0.1.0-dev-0.0.2"

# each CHANGED owned submodule (only if its pointer advanced),
#   same prefix:
git -C <repo>/submodules/<changed> tag -a helix_proxy-0.1.0-
    dev-0.0.2 \
    -m "helix_proxy 0.1.0-dev-0.0.2"
```

The tag string **MUST** be identical across the main repo and every changed owned submodule for this release (§11.4.151) so `git tag -l 'helix_proxy-*` enumerates the whole release surface.

Step 6 — Push tag + main FF-only to all remotes (§2.1 / §11.4.113)

```
for r in github origin upstream; do
  git -C <repo> push "$r" main           # fast-forward only
  git -C <repo> push "$r" helix_proxy-0.1.0-dev-0.0.2
done
```

NEVER --force / --force-with-lease / +<ref> (§11.4.113 — force-push is absolutely forbidden; every push here is a fast-forward because the release commit descends from every mirror tip). If a remote rejects non-FF, return to Step 0 for that remote, merge its new tip, re-validate, re-push. Add the matching push loop for each changed owned submodule to ITS upstream(s). Add the GitLab remote to this loop once OB-1 is resolved.

Step 7 — Publish the GitHub release (gh release create)

```
gh release create helix_proxy-0.1.0-dev-0.0.2 \
  --repo HelixDevelopment/HelixProxy \
  --title "helix_proxy 0.1.0-dev-0.0.2" \
  --notes-file <changelog-section-extract.md> \
  --verify-tag           # tag must already exist (Step 5/6); no
                          auto-create
```

Use --notes-file pointing at the extracted new CHANGELOG.md section. --verify-tag ensures the annotated tag pushed in Step 6 is used rather than gh creating a lightweight tag.

Step 8 — Publish the GitLab release (glab release create)

Blocked until OB-1/OB-2 resolved. Once a GitLab project + reachable auth exist:

```
glab release create helix_proxy-0.1.0-dev-0.0.2 \
  --repo <group>/<project> \
  --name "helix_proxy 0.1.0-dev-0.0.2" \
  --notes-file <changelog-section-extract.md> \
  --ref main
```

The GitLab tag string **MUST** equal the GitHub tag string (§11.4.151 — one release, one prefixed name across every repository it spans). Ensure the tag was pushed to the GitLab remote (Step 6) before / as part of this.

Step 9 — Post-release verification

```
git -C <repo> tag -l 'helix_proxy-*' # new tag
      present
gh release view helix_proxy-0.1.0-dev-0.0.2 --repo
      HelixDevelopment/HelixProxy
glab release view helix_proxy-0.1.0-dev-0.0.2 --repo <group>/
      <project> # after OB-1
```

Confirm the tag on all remotes, both releases published, changelog + exports in sync (§11.4.60/§11.4.65), and the CONTINUATION / session-resumption doc updated (§12.10 / §11.4.131).

Prerequisites / OPERATOR-BLOCKED

Items the operator must provide or decide before a BOTH-CLI (GitHub + GitLab) release can complete. Nothing here is guessed; UNKNOWN is stated where it cannot be verified read-only.

- **OB-1 — GitLab mirror remote + project (BLOCKING for GitLab publish).** `git remote -v` shows only GitHub (three SSH aliases to HelixDevelopment/HelixProxy). There is **no GitLab remote** and the GitLab project path for this repo is **UNKNOWN**. Operator must supply the GitLab `<group>/<project>` (e.g. under a vasic-digital / HelixDevelopment GitLab group) and add it as an SSH remote so §2.1 multi-upstream push + `glab release create --repo <group>/<project>` can target it. Until then the GitLab half of the release cannot proceed. GitHub-only release is unaffected.
- **OB-2 — GitLab auth persistence (verify).** `glab` is authed **via the GITLAB_TOKEN environment variable** (not keyring/config). Confirm this env var is present in the environment that will run the release (a wrapper such as `op plugin run -- glab` may be injecting it). If it is not persistently available, `glab release create` will fail at release time even though the audit showed authed. **UNKNOWN** whether the token scope permits release creation on the target project (cannot verify without the project — OB-1).
- **OB-3 — Version-line discontinuity (decision).** `CHANGELOG.md` documents a 1.x semver line (latest [1.2.0] - 2026-04-23) while the tag line is now 0.1.0-dev-0.0.x. There is no changelog entry for 0.1.0-dev-0.0.1. Operator must confirm the canonical version narrative (is 0.1.0-dev a reset/new track superseding 1.2.0?) and whether legacy v1.1.0/v1.2.0 tags remain or are deprecated, so the new changelog section is accurate.

- **OB-4 — Submodule release-tag prefix (decision, §11.4.151).** The owned submodules (docs_chain, challenges, helix_qa, containers, constitution) currently carry their own helixcode-v1.1.0-* release line — a DIFFERENT prefix than helix_proxy. §11.4.151 requires the SAME prefix across main repo + owned submodules "in one release," but these submodules are shared across projects with their own release cadence. Operator must decide whether this release stamps changed submodules with a helix_proxy-<VERSION> tag (co-existing with helixcode-*) or the project references them by pinned pointer only. Do NOT tag submodules until decided.
- **OB-5 — .env / HELIX_RELEASE_PREFIX (optional hardening).** No .env exists; the prefix resolves via the dir-name fallback (helix_proxy). Functionally READY, but for an authoritative prefix (§11.4.151 order 1), create a gitignored .env with HELIX_RELEASE_PREFIX=helix_proxy and document it in the tracked .env.example (§11.4.30 / §11.4.77).
- **OB-6 — §11.4.40 full-suite retest (BLOCKING, not yet run).** This audit did not run the retest suite (out of read-only scope). The release tag MUST NOT be created until the §11.4.40 complete retest on a clean baseline is GREEN with captured evidence. This is the primary gate independent of GitHub/GitLab readiness.
- **OB-7 — Changelog HTML/PDF exports (§11.4.65).** Root CHANGELOG.md has no .html/.pdf siblings (owned submodules do). If the export mandate applies to the root changelog for this doc class, generate the siblings when the new release section lands. UNKNOWN whether root CHANGELOG.md is in the project's §11.4.65 export scope — confirm with the project's export config.
- **NOTE — this runbook's own exports.** Per §11.4.44/§11.4.65 this file should gain RELEASE_RUNBOOK.html + .pdf siblings; not generated here (task scope = one Markdown doc; no commit/export tooling run).